Fund Management System — Requirements Specification

Document type: Functional & workflow requirements **Derived from:** Design session transcript **Sections:** 6 modules covering the full system lifecycle

---

Table of contents

---

1. System account structure

1.1 Master accounts

Account	Purpose
Master Bank Account	Holds cash received via bank statement imports
Master Cash Account	Holds cash currently managed by the system
Master Fund Account	Holds the total value of the fund

1.2 Member accounts

Account	Purpose
Member Cash Account	Cash belonging to the individual member
Member Fund Account	Member's accumulated contributions to the fund

1.3 Master account invariant

The following equations must hold at all times and are asserted on every nightly batch run:

```
Master Fund Account balance = Σ(all Member Fund Account balances)
                            + Σ(all BACKDATED_DUE obligations not yet collected)

Master Cash Account balance = Σ(all Member Cash Account balances)
```

Any breach of these invariants is classified as a `MASTER_IMBALANCE` and treated as a critical reconciliation failure.

1.4 Member funding paths

Members may fund their cash account via two routes:

Path A — Direct deposit to Master Cash Account - Cash is deposited directly and immediately posted to the member's cash account. - Later cleared via a matching transaction with the Master Bank Account.

Path B — Bank import to Master Bank Account - Bank statement line is imported to the Master Bank Account. - Reflected into the Master Cash Account. - Subsequently posted to the member's cash account.

2. Monthly contribution collection cycle

2.1 Cycle phases

Phase 1 — Cycle initialisation (Day 1)

- Compute each member's contribution amount for the cycle.
- Lock amounts into a pending contribution ledger (immutable snapshot — basis for all late fee calculations).
- Generate collection notices to all members containing: amount due, due date, grace period end date, and each late fee tier threshold.
- Set member cycle status to `PENDING`.
- Snapshot each member's cash account balance at cycle open.

Phase 2 — Collection window (Days 1–N) **Auto-debit — sufficient balance**

If Member Cash Account balance contribution due, debit immediately on Day 1 or the configured debit date.

	Account	Amount
DR	Member Cash Account	Contribution amount
CR	Master Fund Account	Contribution amount
CR	Member Fund Account	Contribution amount (memo)

Status → `COLLECTED`

Auto-debit — insufficient balance

If balance < contribution due, debit the available balance and mark the remaining shortfall as `PARTIALLY_PENDING`. Continue monitoring for incoming deposits.

	Account	Amount
DR	Member Cash Account	Available balance
CR	Master Fund Account	Available balance
CR	Member Fund Account	Partial amount (memo)

Real-time deposit matching

Any deposit arriving during the window triggers an immediate re-evaluation. If the shortfall is covered, post the remainder and mark `COLLECTED`.

> Every deposit to a member's cash account must trigger an immediate check against any outstanding contribution shortfall before the deposit becomes available for other uses.

Phase 3 — Grace period close & late fee engine (Day N+1 onward) At close of the collection window, flag all members with outstanding balances as `OVERDUE`. Record `overdue_since` timestamp — this is the immutable clock start for all late fee tier calculations.

> The `days_overdue` counter starts from the exact close-of-business timestamp of the collection window, not the cycle start date.

Tiered late fee schedule

Days overdue	Status	Action
1–3	OVERDUE	Reminders only — no fee applied
After day 3	LATE — Tier 1	Apply Fee X — post to member cash account
After day 10	LATE — Tier 2	Apply Fee Y (replaces Tier 1)
After day 20	LATE — Tier 3	Apply Fee Z (replaces Tier 2) — escalate, flag account

Late fee journal entry

	Account	Amount
DR	Member Cash Account	Late fee amount
CR	Late Fee Income Account	Late fee amount

Late fee model options (configurable)

- `replacement` — each new tier reverses the prior fee and posts the new one.
- `cumulative` — each new tier is posted as an additional charge on top of prior fees.

Nightly late fee batch algorithm

```
FOR each member WHERE status IN ('OVERDUE', 'LATE_T1', 'LATE_T2', 'LATE_T3'):
  days     = today - overdue_since_date
  new_tier = lookup_tier(days)
  IF new_tier != member.current_tier:
    reverse_prior_fee_entry(member)    // if replacement model
    post_late_fee(member, new_tier.fee)
    update_status(member, new_tier.label)
    notify_member(member)
```

Phase 4 — Settlement & bank reconciliation **Bank import clearing**

	Account	Amount
DR	Master Bank Account	Deposit amount
CR	Master Cash Account	Deposit amount

Direct cash deposit

	Account	Amount
DR	Master Cash Account	Deposit amount
CR	Member Cash Account	Deposit amount

At month-end, reconcile all three master accounts. Unresolved shortfalls carry into the next cycle with accumulated late fees.

Phase 5 — Reporting & audit trail

- Per-member collection summary: amount due, collected, outstanding, fees applied, status.
- Immutable audit log: every journal entry, status change, fee application, and deposit match — timestamped with operator ID.

2.2 Member cycle status state machine

```
PENDING
    balance covers → COLLECTED
    partial balance → PARTIALLY_PENDING
         deposit covers shortfall → COLLECTED
         window closes → OVERDUE
    window closes (zero balance) → OVERDUE
         day 3+  → LATE Tier 1
               day 10+ → LATE Tier 2
                     day 20+ → LATE Tier 3 (escalate)

Any OVERDUE / LATE state:
    payment received → SETTLING → COLLECTED
```

---

3. Loan lifecycle

3.1 Loan eligibility

All gates are evaluated in sequence. Any failure rejects the request unless an admin override is applied.

Gate	Rule	Admin override?
Tenure	Member must have X years of active membership	Yes — mandatory reason
Minimum fund balance	Member Fund Account balance Y amount	Yes — mandatory reason
Delinquency	Member must have no active delinquency flag	Yes — mandatory reason
Active loan count	Member may not exceed X simultaneous active loans (typically 1–2)	Yes — mandatory reason
Borrow limit	Requested amount X × Member Fund Account balance	Yes — mandatory reason
Guarantor	A valid guarantor must be named — active, in good standing, not the borrower	No — cannot be overridden

3.2 Loan request submission & queue management

1. Member submits: requested amount, loan type (standard or emergency), guarantor ID, grace period election (0, 1, or 2 cycles).
2. System snapshots member's fund balance and active loan count at submission time.
3. All eligibility gates are evaluated. Any override is tagged and logged.
4. System assigns the loan to an EMI tier based on requested amount.
5. Standard requests enter the FIFO queue ordered by submission timestamp.
6. Emergency requests are inserted at the front of the queue, ahead of all pending standard requests. Multiple emergencies are FIFO among themselves.
7. Status → `PENDING_ADMIN`.

```
FUNCTION insert_to_queue(request):
  IF request.type == 'EMERGENCY':
    position = front_of_queue()
  ELSE:
```

```
    position = end_of_queue()
  queue.insert(request, position)
  notify(member, guarantor, position)
  set_status(request, 'PENDING_ADMIN')
```

3.3 EMI tier structure (configurable)

Loan amount range	EMI
1K – 10K	1K
11K – 30K	1.5K
31K – 60K	2.5K
Emergency	Configurable per request

3.4 Fund tier structure

Fund tiers define the percentage of the Master Fund Account available to fund loans at each loan tier. Tiers are configurable from 0%–100% and may overlap.

Fund tier	Linked loan tier	Allocation	Overlaps?
Fund Tier A	Loan Tier 1 (1K–10K)	0–100%	Yes
Fund Tier B	Loan Tier 2 (11K–30K)	0–100%	Yes
Fund Tier C	Loan Tier 3 (31K–60K)	0–100%	Yes
Fund Tier E (Emergency)	Emergency requests only	0–100%	Yes

Loan allocation split

Portion	Definition
Member's portion	Member Fund Account balance (up to loan amount)
Master's portion	Loan amount − Member's portion
Repayment threshold	Master portion + X% of requested loan amount

The loan is considered fully repaid when the master portion + settlement threshold has been repaid.

Fund availability check

```
FUNCTION check_fund_availability(loan_tier, amount):
  tier_pct  = config.fund_tier[loan_tier].allocation_pct
  tier_pool = master_fund_balance * tier_pct
  committed = sum of all active disbursements against this tier
  available = tier_pool - committed
  RETURN available >= amount   // full approval
      OR available > 0          // partial disbursement possible
```

3.5 Approval & disbursement

1. Admin reviews the next request in queue: member, amount, tier, fund tier availability, override flags.
2. System checks fund tier availability.
   - Sufficient funds → full approval.
   - Partial funds → admin may disburse partial amount.
   - No funds → request deferred or rejected.

3. Admin records decision. Partial approval requires specifying the disbursed amount. Rejection requires a reason.
4. Each disbursement tranche posts immediately:

	Account	Amount
DR	Master Fund Account	Disbursed amount
CR	Member Cash Account	Disbursed amount
Memo	Member Fund Account	−Disbursed amount (reflected)

5. The loan is NOT active until the total approved amount is fully disbursed. Partial tranches accumulate under status `PARTIALLY_DISBURSED`.
6. On full disbursement → status = `ACTIVE`. Repayment schedule is built.

3.6 Grace period

Elected at submission time. Locked after approval. Cannot be changed.

Election	Effect
0 cycles	First EMI due in the immediate next collection cycle
1 cycle	First EMI due in cycle N+2
2 cycles	First EMI due in cycle N+3

Grace cycles are interest-free deferral. No EMI and no late fee during grace. Contribution exemption applies during grace.

3.7 Repayment schedule construction

```
FUNCTION build_schedule(loan):
  emi          = lookup_emi_tier(loan.amount)
  threshold    = loan.master_portion + (loan.amount * settlement_pct)
  grace_cycles = loan.grace_election
  start_cycle  = current_cycle + 1 + grace_cycles
  cycle        = start_cycle
  total_repaid = 0
  schedule     = []

  WHILE total_repaid < threshold:
    schedule.append({cycle, emi, status: 'PENDING'})
    total_repaid += emi
    cycle        += 1

  schedule[-1].amount = threshold - (total_repaid - emi)  // adjust last instalment
  loan.repayment_schedule = schedule
  RETURN schedule
```

EMI collection uses the identical mechanism as contribution collection — including auto-debit, partial debit, real-time deposit matching, and tiered late fees.

EMI collection journal

	Account	Amount
DR	Member Cash Account	EMI amount

	Account	Amount
CR	Master Fund Account	EMI amount
CR	Loan Repayment Ledger	EMI amount (memo)

3.8 Early settlement

Full early settlement

Outstanding balance = `threshold - total_repaid_to_date`. Member must fund their cash account with this amount.

	Account	Amount
DR	Member Cash Account	Remaining threshold
CR	Master Fund Account	Remaining threshold

Loan status → `REPAID`. Contribution exemption lifted next cycle. Guarantor liability released.

Partial early settlement

Member specifies a partial amount 1 full EMI, then elects a schedule treatment:

- Option A — Roll-up (compress): upcoming EMI cycles fully covered are marked SETTLED; schedule shortens.
- Option B — Skip cycles: upcoming cycles are skipped; original schedule length preserved with gaps inserted.

```
FUNCTION apply_partial_settlement(loan, amount, option):
  cycles_covered = floor(amount / loan.emi)

  IF option == 'ROLLUP':
    mark next cycles_covered pending EMIs as SETTLED
    rebuild remaining schedule from (current + cycles_covered + 1)

  IF option == 'SKIP':
    mark next cycles_covered pending EMIs as SKIPPED
    resume EMIs from (current + cycles_covered + 1)

  loan.total_repaid += amount
  IF loan.total_repaid >= loan.threshold:
    close_loan(loan)
```

3.9 Contribution exemption

A member under an active loan repayment schedule (including grace period) is exempt from monthly contributions.

Condition	Contribution required?
Loan ACTIVE, in repayment	Exempt
Loan in grace period	Exempt
Loan PARTIALLY_DISBURSED	Exempt
Loan REPAID this cycle	Due from next cycle
Loan transferred to guarantor — original borrower	Due (suspension lifts exemption)
Loan transferred to guarantor — guarantor	Exempt
Multiple active loans	Exempt (one active loan is sufficient)

Condition	Contribution required?

```
MONTHLY_COLLECTION_JOB:
  FOR each member:
    IF has_active_loan(member) OR in_grace_period(member):
      skip_contribution_debit(member)
    ELSE:
      proceed_with_contribution_collection(member)
```

3.10 Guarantor liability & delinquency escalation

Escalation ladder

Missed EMIs	Action	Borrower status
1 – (X−1)	Late fees apply per collection cycle rules	Overdue
X missed	Guarantor formally notified — liability warning issued	Delinquent
Y missed	Guarantor assumes full liability; loan transferred	Suspended

Guarantor liability scope

The guarantor is liable only for:

```
Guarantor obligation = (Master portion of original loan)
                     - (repayments already credited to master portion)
                     + (X% of original requested loan amount)
```

The member's own fund equity portion is excluded.

Loan transfer flow

1. Loan ownership re-assigned to guarantor. Guarantor becomes borrower of record.
2. Original borrower status → `SUSPENDED`. Cannot request loans, make withdrawals, or submit admin-approval transactions. Cannot be a guarantor on any other loan.
3. New repayment schedule built for guarantor based on remaining obligation. Counts against guarantor's active loan limit.
4. Reinstatement of suspended borrower requires explicit admin action — not automatic.

```
NIGHTLY_JOB: evaluate_missed_emis()
  FOR each active loan:
    missed = count EMIs with status MISSED or OVERDUE
    IF missed == X:
      notify(guarantor, 'liability_warning')
      set_borrower_status('DELINQUENT')
    IF missed >= Y:
      transfer_loan(loan, new_owner=guarantor)
      suspend_member(loan.original_borrower)
      rebuild_schedule(guarantor, calc_guarantor_liability(loan))
      notify(guarantor, 'liability_active')
      notify(borrower, 'account_suspended')
```

3.11 Loan status state machine

```
PENDING_ADMIN
     rejected → REJECTED (terminal)
     approved (partial) → PARTIALLY_DISBURSED
           fully disbursed → ACTIVE
                 (grace period) → ACTIVE [no EMI due yet]
                 in repayment → ACTIVE
                       full early settlement → REPAID (terminal)
                       partial early settlement → ACTIVE [schedule rebuilt]
                       X missed EMIs → DELINQUENT [guarantor warned]
                       Y missed EMIs → TRANSFERRED [guarantor owns loan]
                             guarantor repays fully → REPAID (terminal)
                 all EMIs collected → REPAID (terminal)
```

---

4. Migration & historical cycle resolution

4.1 Rationale

Members migrated from a legacy system may have a join date extending far into the past, with no contribution data available. The new system must establish opening balances and resolve all historical cycles without triggering delinquency rules prematurely.

4.2 Member onboarding — opening balances

Opening cash balance

	Account	Amount
DR	Master Cash Account	Cash opening balance
CR	Member Cash Account	Cash opening balance

Opening fund balance

	Account	Amount
DR	Master Fund Account	Fund opening balance
CR	Member Fund Account	Fund opening balance

Both entries are tagged `MIGRATION_OPENING_BALANCE` with the migration effective date (not the posting date).

4.3 Migration cutoff date

- Record `migration_cutoff_date` per member — the date from which the new system takes over.
- This date is immutable once set. Admin approval required to amend.
- The member's join date and cutoff date together define the historical window.

4.4 Historical cycle stub generation

- System generates one cycle record per month between `join_date` and `migration_cutoff_date`.
- Each stub is created with status `UNRESOLVED` — **not** `MISSED`.
- Member profile status → `MIGRATION_PENDING`.

- While `MIGRATION_PENDING`: late fee engine suppressed, delinquency rules inactive, guarantor notifications blocked.

```
FUNCTION generate_historical_stubs(member):
  cycle = first_cycle_month(member.join_date)
  WHILE cycle <= member.migration_cutoff_date:
    stubs.append({
      cycle_date:      cycle,
      status:          'UNRESOLVED',
      origin:          'MIGRATION',
      amount_due:      lookup_contribution_rate(cycle),
      late_fee_exempt: true
    })
    cycle = next_month(cycle)
  member.status = 'MIGRATION_PENDING'
```

> Do not mark historical cycle stubs as MISSED at creation. This would trigger the late fee engine and delinquency flags before any resolution process runs.

4.5 Historical cycle classification

Admin classifies each stub (individually or in batch by date range):

Classification	Meaning	System action
WAIVED	Predates available records; admin grants waiver	Closed — zero obligation, audit log only
BACKDATED_PAID	Evidence of payment in old system	Marked settled — no additional debit
BACKDATED_DUE	Genuinely missed; obligation exists	Converted to payable — resolution method required
OB_ABSORBED	Opening balance already represents these contributions	Batch-closed — no debit, opening balance flagged as settlement
ESCALATED	Disputed or under investigation	Frozen — does not count toward delinquency

> Use `OB_ABSORBED` only when the opening balance was computed from cumulative contribution totals in the old system. If the opening balance was an estimate, use `WAIVED` and note the basis in the audit record.

4.6 Resolution methods for BACKDATED_DUE cycles

Method A — Lump-sum settlement

	Account	Amount
DR	Member Cash Account	Total backdated amount
CR	Master Fund Account	Total backdated amount
CR	Member Fund Account	Total backdated amount (memo)

Method B — Instalment plan

Total backdated amount divided into equal instalments over N future cycles. Runs alongside regular contributions (no automatic exemption). Late fee rules apply to missed instalments.

```
FUNCTION build_instalment_schedule(member, due_stubs):
  total_due   = sum(s.amount_due FOR s IN due_stubs)
  n           = admin_config.migration_instalment_cycles
  instalment  = ceil(total_due / n)
  start_cycle = next_active_cycle()
  FOR i IN range(n):
    amt = instalment IF i < n-1
          ELSE total_due - (instalment * (n-1))
    schedule.append({cycle: start_cycle+i, amount: amt,
                     status: 'PENDING', type: 'MIGRATION_INSTALMENT'})
```

Method C — Opening balance offset

Applicable only when opening fund balance total backdated obligations. Member consent required.

	Account	Amount
DR	Member Fund Account	Total backdated amount
CR	Master Fund Account	Total backdated amount

Late fees for backdated cycles

Late fees are never automatically applied to `BACKDATED_DUE` cycles regardless of outstanding duration. Manual admin override required if fees are deemed appropriate; override reason must be logged.

4.7 Delinquency clearance

Mandatory conditions (all must be met before clearance)

Condition	Required?
No UNRESOLVED stubs remain	Mandatory
All BACKDATED_DUE cycles have a resolution method assigned	Mandatory
No ESCALATED cycles remain open	Mandatory
Lump-sum or offset fully posted (if Method A or C)	Conditional
Instalment schedule built and first date set (if Method B)	Conditional
Admin sign-off	Mandatory

Clearance flow

1. System validates all mandatory conditions. Unmet items block the clearance action.
2. Admin reviews resolution summary and submits approval.
3. Member status → `ACTIVE`. Normal operating rules apply from the next cycle.
4. Member notified with migration clearance summary.

Partial clearance (advanced)

For members with very long histories, admin may grant `PARTIAL_CLEARANCE_GRANTED`: `ACTIVE` status is issued for all new-cycle operations while a subset of `ESCALATED` cycles continues to be investigated in the background.

4.8 Migration journal entry tags

Entry tag	Event
`MIGRATION_OPENING`	Opening balance (cash or fund)
`MIGRATION_WAIVER`	Cycle waiver — audit log only, no cash movement
`MIGRATION_BACKDATED_PAID`	Cycle marked settled — no cash movement
`MIGRATION_OB_ABSORBED`	Cycle absorbed into opening balance — no cash movement
`MIGRATION_LUMPSUM`	Lump-sum backdated settlement
`MIGRATION_INSTALMENT`	Individual instalment collected
`MIGRATION_OB_OFFSET`	Opening balance offset against backdated obligation
`MIGRATION_MANUAL_FEE`	Manual late fee override (admin-only)

All migration journal entries must carry both `effective_date` (the historical month) and `posting_date` (actual date of entry).

---

5. Reconciliation workflow

5.1 Architecture

The reconciliation control layer sits above all operational processes as a continuous validation mechanism. It does not replace any workflow — it validates, detects drift, and routes discrepancies to automatic resolution or a human queue.

Two execution modes:

- Nightly batch — full sweep of all domains at a fixed time before any operational posting.
- Real-time event trigger — fires on every transaction posted; validates double-entry integrity and exemption rules immediately.

5.2 Discrepancy classification

Every detected discrepancy is classified before routing:

Type	Description
Timing difference	Entry exists on one side; counterpart expected within N days
Amount mismatch	Entry exists on both sides but amounts differ
Missing entry	One side of a journal entry is absent
Duplicate entry	Same transaction posted more than once
Status mismatch	Account or cycle status inconsistent with posted entries

5.3 Auto-resolve tolerance

Discrepancies within the configured tolerance (e.g. 0.01) are automatically resolved via a rounding adjustment entry. Discrepancies exceeding tolerance that match a known in-flight transaction are deferred 24h with status `TIMING_DIFFERENCE`. If unresolved after 48h, they escalate to the manual queue.

5.4 Domain 1 — Master account reconciliation

- Run first in every nightly batch. A `MASTER_IMBALANCE_UNRESOLVED` halts all further batch posting.
- Auto-resolve: rounding differences tolerance.
- Auto-resolve: timing differences with matched in-flight transaction (defer 24h, re-check).

- Manual: all unexplained imbalances exceeding tolerance.

Rounding adjustment journal

	Account	Amount
DR	Reconciliation Suspense Account	Delta amount
CR	Master Fund / Cash Account	Delta amount

5.5 Domain 2 — Contribution reconciliation

Per-member, per-cycle validation run at end of each collection window and again 48h later.

Check	Auto-resolve?	Manual trigger?
Member debited but Master Fund not credited	Yes — post missing CR leg	If member fund memo also missing
Master Fund credited but member not debited	No	Always — orphan credit
Amount collected amount due (within tolerance)	Yes	If delta > tolerance
Cycle marked COLLECTED but debit not posted	No	Always — status mismatch
Cycle marked PENDING past window close	Yes — re-run debit attempt	If debit fails again
Duplicate debit for same cycle	No	Always
MIGRATION_PENDING member debited for live cycle	Yes — reverse debit	If reversal fails
Contribution collected from loan-exempt member	Yes — reverse collection	Always notify member

Exempt reversal journal

	Account	Amount
CR	Member Cash Account	Contribution refunded
DR	Master Fund Account	Contribution refunded

5.6 Domain 3 — EMI & loan disbursement reconciliation

EMI checks

Check	Auto-resolve?	Manual trigger?
EMI collected but loan repayment ledger not updated	Yes — post memo	If threshold recalculation affected
EMI marked MISSED but cash had sufficient funds	No	Always
Total collected exceeds loan repayment threshold	Yes — refund overpayment	If over-collection > 1 EMI
Loan ACTIVE but all EMIs marked COLLECTED	Yes — close loan	Never (automatic)
Guarantor and borrower both debited for same cycle	No	Always — duplicate

Check	Auto-resolve?	Manual trigger?
Grace cycle has an EMI debit	Yes — reverse debit	If reversal produces imbalance

Disbursement checks

Check	Auto-resolve?	Manual trigger?
Master Fund debited but member cash not credited	No	Always
Member Fund memo not reflected on disbursement	Yes — post memo correction	If affects allocation calc
Loan ACTIVE but disbursed amount < approved amount	No	Always — premature activation
Fund tier over-committed	No	Always
Repayment schedule built before full disbursement	Yes — void and rebuild	If any EMIs already collected

Over-collection refund journal

	Account	Amount
CR	Member Cash Account	Overpaid amount
DR	Master Fund Account	Overpaid amount

5.7 Domain 4 — Bank clearing reconciliation

Automated matching algorithm

```
FOR each bank_statement_line imported today:
  candidates = find_cash_account_entries(
    amount     = bank_line.amount ± tolerance,
    date_range = bank_line.date ± 3 days,
    status     = 'PENDING_CLEARANCE'
  )
  IF candidates.count == 1:
    auto_match(bank_line, candidates[0])
    post_clearing_entry(bank_line, candidates[0])
    status = 'CLEARED'
  ELIF candidates.count > 1:
    raise RECON_AMBIGUOUS_MATCH → manual queue
  ELIF candidates.count == 0:
    raise RECON_UNMATCHED_BANK_LINE → manual queue
```

Clearing journal

	Account	Amount
DR	Master Bank Account	Deposit amount
CR	Master Cash Account	Deposit amount

Bank clearing exception types

Exception	Likely cause	Admin action
UNMATCHED_BANK_LINE	Unknown depositor	Identify, post to correct member account, clear
UNMATCHED_CASH_ENTRY	No bank line received	Verify with bank, void or adjust
AMBIGUOUS_MATCH	Multiple same-amount pending entries	Disambiguate via reference number
AMOUNT_MISMATCH	Bank and system amounts differ	Post adjustment entry, clear original
STALE_PENDING	Cash entry pending > 30 days	Investigate, void, or request bank trace

Direct cash deposits unmatched by a bank line within N days raise `CASH_DEPOSIT_UNBANKED`.

5.8 Domain 5 — Late fee reconciliation

Check	Auto-resolve?	Manual trigger?
Fee applied at wrong tier	Yes — reverse and repost correct tier	If boundary-case disputed
Fee applied to migration cycle	Yes — always auto-reversed	Never
Fee applied to loan-exempt member	Yes — reverse	If exemption disputed
Fee applied to grace-period EMI	Yes — always auto-reversed	Never
Fee posted to wrong account	No	Always
Replacement model: prior tier not reversed	Yes — reverse prior, repost current	If prior already collected in cash
Fee Income Account Σ(all posted fees)	No	Always

Fee tier correction journal (replacement model)

	Account	Amount
CR	Member Cash Account	Wrong tier fee (reverse)
DR	Late Fee Income Account	Wrong tier fee (reverse)
DR	Member Cash Account	Correct tier fee
CR	Late Fee Income Account	Correct tier fee

5.9 Domain 6 — Migration reconciliation

Check	Auto-resolve?	Manual trigger?
Opening balance entry missing one leg	No	Always
Σ(MIGRATION_OPENING) master snapshot at migration date	No	Always
Member has opening entry but no cutoff date	Partial — flag	Admin must set date
OB_OFFSET causes negative member fund balance	No	Always

Check	Auto-resolve?	Manual trigger?
UNRESOLVED stubs for a member with ACTIVE status	No	Always
Instalment total > total backdated obligation	Yes — refund excess	If excess > 1 instalment
ACTIVE member debited for WAIVED cycle	Yes — reverse debit	Never

Migration ledger integrity assertion

```
ASSERT: Σ(MIGRATION_OPENING fund entries)
       + Σ(MIGRATION_LUMPSUM or INSTALMENT collected)
       + Σ(MIGRATION_OB_OFFSET)
       == expected total fund obligation for all migrated members
```

5.10 Manual exception queue

Exception record fields

Field	Description
`exception_id`	Unique, immutable
`exception_type`	Typed code (e.g. `RECON_UNMATCHED_BANK_LINE`)
`domain`	master_account · contribution · emi · loan · bank_clearing · late_fee · migration
`severity`	CRITICAL · HIGH · MEDIUM · LOW
`amount_delta`	Monetary discrepancy
`affected_entities`	member_id, cycle_id, loan_id, transaction_id
`auto_resolve_attempted`	Boolean + reason
`raised_at`	Detection timestamp
`sla_deadline`	Computed from severity
`assigned_to`	Admin user
`resolution`	Action, journal entries, outcome
`resolved_at`	Resolution timestamp

Severity SLA

Severity	SLA	Batch behavior
CRITICAL	Immediate	Batch halted until resolved
HIGH	Same business day	Affected member transactions held
MEDIUM	48 hours	Normal batch continues
LOW	Next cycle	Normal batch continues

Admin resolution actions

Action	Description
Post correction entry	Admin selects corrective journal type; system validates it resolves the delta
Reverse transaction	Exact reversal of original; mandatory reason code required

Action	Description
Reclassify	Change transaction or cycle classification; old and new classification logged
Write-off	Post to write-off account; only available for LOW and MEDIUM severity
Escalate	Move to higher severity; SLA resets; reason mandatory
Override and accept	Close exception without corrective entry; supervisor sign-off required

5.11 Nightly batch orchestrator

```
NIGHTLY_RECON_BATCH():
  snapshot = take_balance_snapshot(timestamp=now())

  // Step 1 - Master invariant (hard gate)
  result = check_master_invariant(snapshot)
  IF result == CRITICAL_FAIL:
    halt_all_posting()
    raise_exception(MASTER_IMBALANCE_UNRESOLVED, CRITICAL)
    RETURN

  // Step 2 - Domain checks (parallel)
  run_in_parallel([
    reconcile_contributions(snapshot),
    reconcile_emi_and_loans(snapshot),
    reconcile_bank_clearing(snapshot),
    reconcile_late_fees(snapshot),
    reconcile_migration(snapshot)
  ])

  // Step 3 - Auto-resolve
  FOR each exception raised:
    IF exception.auto_resolvable:
      attempt_auto_resolve(exception)
      IF success: log_auto_resolution(exception)
      ELSE:       route_to_manual_queue(exception, severity=ESCALATED)
    ELSE:
      route_to_manual_queue(exception)

  // Step 4 - Final re-assertion
  final_snapshot = take_balance_snapshot(timestamp=now())
  re_check_master_invariant(final_snapshot)
  write_recon_report(snapshot, final_snapshot)
```

5.12 Real-time event trigger

```
EVENT_TRIGGER: on_transaction_posted(txn):
  IF txn.dr_total != txn.cr_total:
    raise_exception(UNBALANCED_ENTRY, CRITICAL)
    void_transaction(txn)
    RETURN
```

```
  IF txn.member.status == 'MIGRATION_PENDING'
     AND txn.type NOT IN MIGRATION_ALLOWED_TYPES:
    raise_exception(INELIGIBLE_ACCOUNT_POSTING, HIGH)
    reverse_transaction(txn)
    RETURN

  IF txn.type == 'LATE_FEE'
     AND (txn.cycle.origin == 'MIGRATION'
          OR txn.member.has_active_loan
          OR txn.cycle.in_grace_period):
    auto_reverse_fee(txn)
    log_auto_resolution('RECON_AUTO_FEE_EXEMPTION_REVERSAL', txn)
    RETURN

  update_running_totals(txn)
  check_member_invariant(txn.member)
```

5.13 Member-level invariant check

```
FUNCTION check_member_invariant(member):
  expected_fund = member.opening_fund_balance
                + Σ(contributions collected)
                + Σ(migration instalments collected)
                - Σ(loan disbursements - member portion)
                + Σ(EMI repayments)

  expected_cash = member.opening_cash_balance
                + Σ(deposits received)
                + Σ(loan disbursements credited)
                - Σ(contributions debited)
                - Σ(EMI debited)
                - Σ(late fees debited)
                - Σ(cash outs)

  IF abs(expected_fund - member.fund_account.balance) > tolerance:
    raise_exception(MEMBER_FUND_DRIFT, MEDIUM, member)

  IF abs(expected_cash - member.cash_account.balance) > tolerance:
    raise_exception(MEMBER_CASH_DRIFT, MEDIUM, member)
```

---

6. Configurable parameters

All parameters below must be stored in a configuration table — not hardcoded — to allow cycle-by-cycle adjustment without a system deployment.

6.1 Contribution collection

Parameter	Description
`collection_window_days`	Length of the monthly collection window (N days)
`late_fee_tier_1_day`	Days overdue before Tier 1 fee applies (e.g. 3)
`late_fee_tier_2_day`	Days overdue before Tier 2 fee applies (e.g. 10)
`late_fee_tier_3_day`	Days overdue before Tier 3 fee applies (e.g. 20)

Parameter	Description
late_fee_tier_1_amount	Fee amount X
late_fee_tier_2_amount	Fee amount Y
late_fee_tier_3_amount	Fee amount Z
late_fee_model	replacement or cumulative

6.2 Loan eligibility

Parameter	Description
min_membership_years	Minimum years of membership before loan eligibility (X)
min_fund_balance	Minimum fund account balance to request a loan (Y)
borrow_multiplier	Maximum loan amount as a multiple of fund balance
max_active_loans	Maximum simultaneous active loans per member (typically 1–2)
settlement_threshold_pct	% of requested loan added to master portion to define full repayment
grace_period_options	Allowed grace period elections (e.g. 0, 1, 2 cycles)

6.3 Loan tiers

Parameter	Description
emi_tiers	Table of loan amount ranges and associated EMI amounts
fund_tier_allocations	Per-tier % of master fund available for each loan tier (0–100%, may overlap)

6.4 Guarantor escalation

Parameter	Description
guarantor_warning_threshold	Missed EMIs before guarantor notification (X)
guarantor_liability_threshold	Missed EMIs before automatic loan transfer (Y)

6.5 Migration

Parameter	Description
migration_instalment_cycles	Number of future cycles over which backdated obligations are spread (Method B)

6.6 Reconciliation

Parameter	Description
`recon_tolerance`	Maximum monetary delta auto-resolved as rounding (e.g. 0.01)
`timing_diff_defer_hours`	Hours before a timing difference is escalated to manual queue (e.g. 48)
`bank_match_date_range_days`	± days around a bank line date used in matching candidates (e.g. 3)
`cash_deposit_unbanked_days`	Days before an unmatched direct cash deposit raises CASH_DEPOSIT_UNBANKED
`stale_pending_days`	Days before an uncleared cash account entry is flagged STALE_PENDING

Appendix A — Master journal entry reference

Event	DR	CR	Tag
Opening cash balance (migration)	Master Cash Account	Member Cash Account	MIGRATION_OPENING
Opening fund balance (migration)	Master Fund Account	Member Fund Account	MIGRATION_OPENING
Contribution collected	Member Cash Account	Master Fund Account + Member Fund (memo)	—
Contribution — exempt reversal	Master Fund Account	Member Cash Account	RECON_EXEMPT_REVERSAL
EMI collected	Member Cash Account	Master Fund Account + Loan Repayment Ledger (memo)	—
EMI over-collection refund	Master Fund Account	Member Cash Account	RECON_EMI_OVERPAYMENT_REF
Loan disbursement	Master Fund Account	Member Cash Account	—
Loan disbursement memo	Member Fund Account (−)	—	—
Full early settlement	Member Cash Account	Master Fund Account	—
Partial early settlement	Member Cash Account	Master Fund Account	—
Late fee applied	Member Cash Account	Late Fee Income Account	—
Late fee tier correction	Member Cash Account + Late Fee Income (reverse & repost)	—	RECON_AUTO_FEE_TIER_CORREC
Bank import clearing	Master Bank Account	Master Cash Account	—
Direct cash deposit	Master Cash Account	Member Cash Account	—
Migration lump-sum settlement	Member Cash Account	Master Fund + Member Fund (memo)	MIGRATION_LUMPSUM
Migration instalment collected	Member Cash Account	Master Fund + Member Fund (memo)	MIGRATION_INSTALMENT
Migration OB offset	Member Fund Account	Master Fund Account	MIGRATION_OB_OFFSET
Reconciliation rounding adj.	Reconciliation Suspense Account	Master Fund / Cash Account	RECON_AUTO_ROUNDING
Reconciliation manual correction	(admin-specified)	(admin-specified)	RECON_MANUAL_CORRECTION

Appendix B — Status codes reference

Code	Domain	Meaning
PENDING	Contribution / EMI	Obligation created, not yet collected
PARTIALLY_PENDING	Contribution / EMI	Partially collected, shortfall remains
COLLECTED	Contribution / EMI	Fully collected
OVERDUE	Contribution / EMI	Past window close, no fee yet (days 1–3)
LATE_T1 / T2 / T3	Contribution / EMI	Fee tier applied
SETTLING	Contribution / EMI	Payment received, processing
PENDING_ADMIN	Loan	Awaiting admin review in queue
PARTIALLY_DISBURSED	Loan	Partial tranche posted, not yet active
ACTIVE	Loan	Fully disbursed, in repayment
DELINQUENT	Loan / Member	Missed EMIs — guarantor warned
TRANSFERRED	Loan	Loan ownership moved to guarantor
REPAID	Loan	Fully repaid — terminal
REJECTED	Loan	Admin rejected — terminal
SUSPENDED	Member	Borrower suspended after loan transfer
MIGRATION_PENDING	Member	Awaiting cycle resolution clearance
ACTIVE	Member	Normal operating status
UNRESOLVED	Migration cycle stub	Not yet classified
BACKDATED_DUE	Migration cycle stub	Obligation confirmed, resolution required
BACKDATED_PAID	Migration cycle stub	Evidence of prior payment — no action
WAIVED	Migration cycle stub	Admin waiver granted
OB_ABSORBED	Migration cycle stub	Covered by opening balance
ESCALATED	Migration cycle stub	Under investigation — frozen
PENDING_CLEARANCE	Bank entry	Awaiting bank import match
CLEARED	Bank entry	Matched and posted
STALE_PENDING	Bank entry	Uncleared beyond threshold